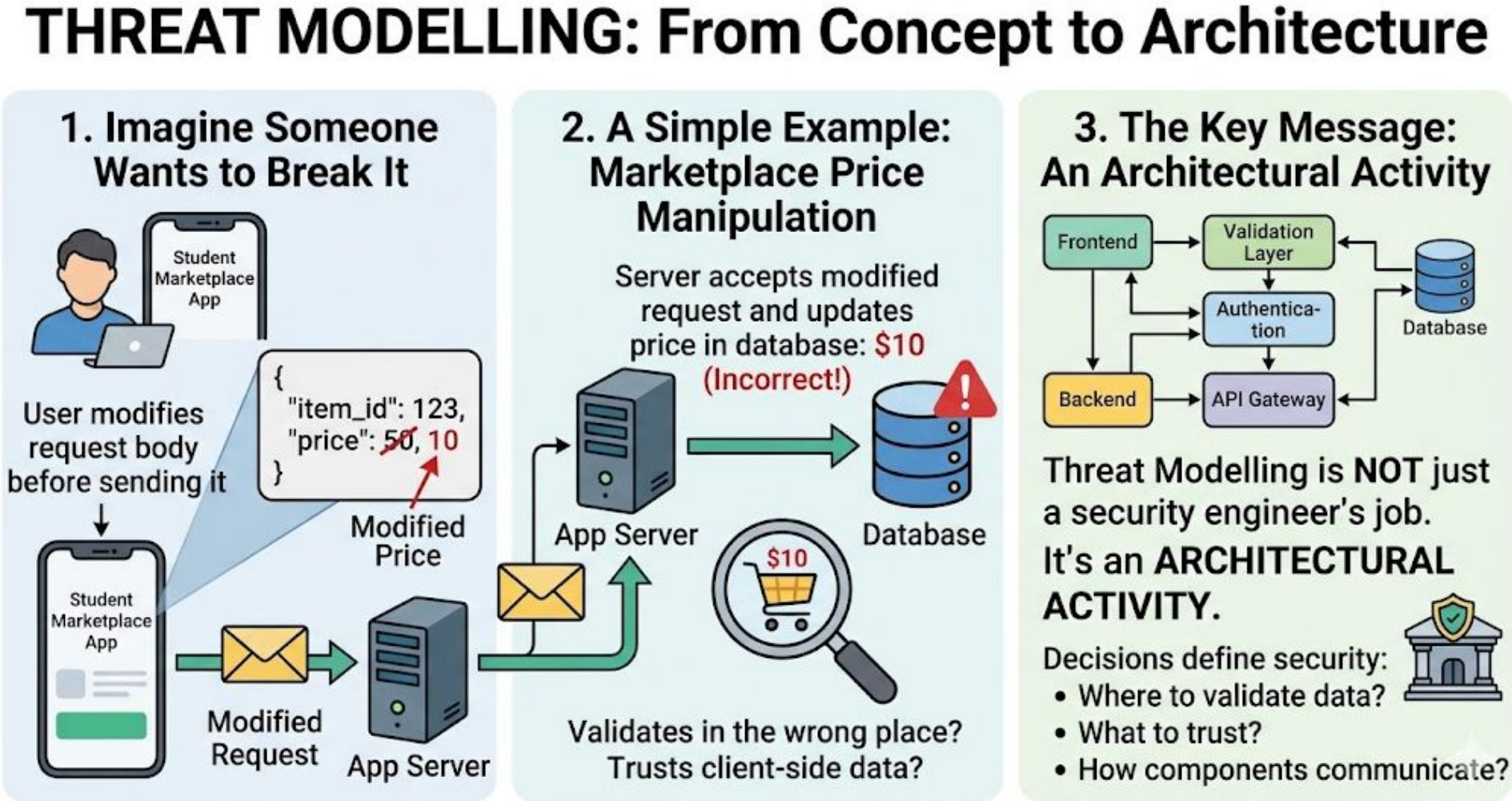
UNIVERSITY OF SZEGED
Department of Software Engineering
UNIVERSITAS SCIENTIARUM SZEGEDIENSIS

Software Architectures

Practice 9

Zoltán Richárd JÁNKI, PhD

Threat Modelling

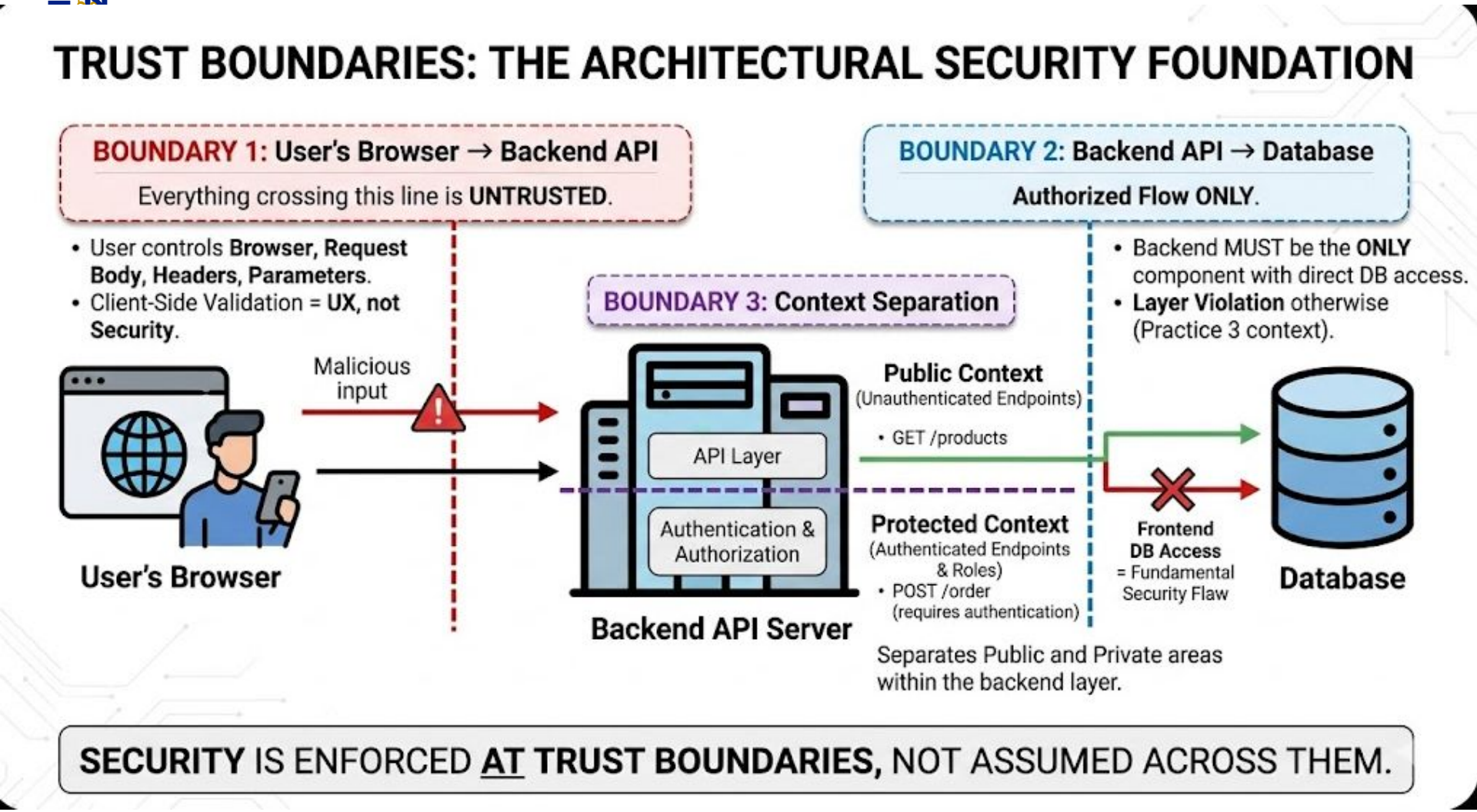


Threat Model

Answer 4 questions:

- **What are we building?** — You already know this from your architecture diagrams, feature decomposition, and OpenAPI spec.
- **What can go wrong?** — This is the creative/analytical part.
- **What are we going to do about it?** — Mitigate, accept, or transfer the risk.
- **Did we do a good enough job?** — Review and iterate.

Trust Boundaries

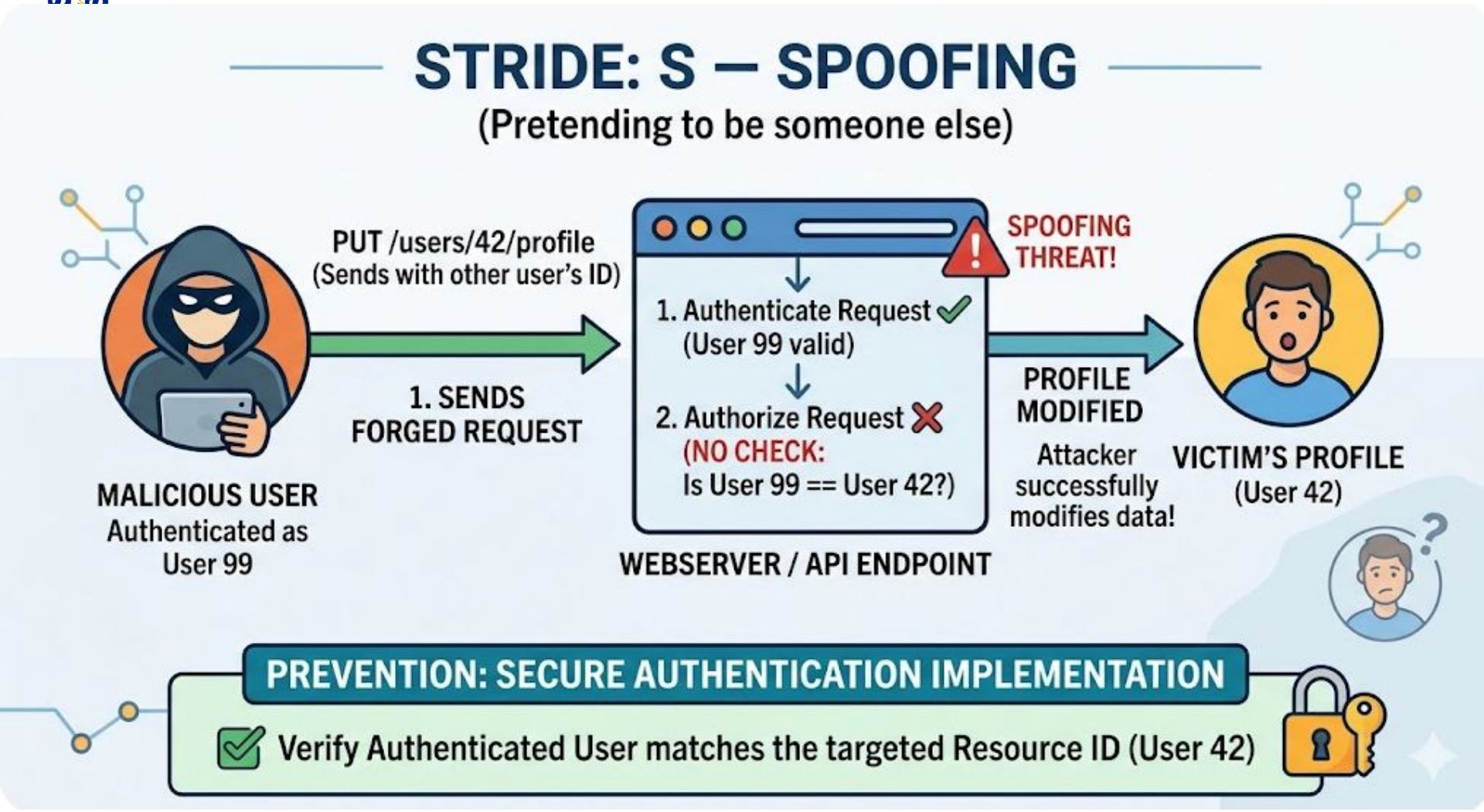


Identifying Threats - STRIDE model

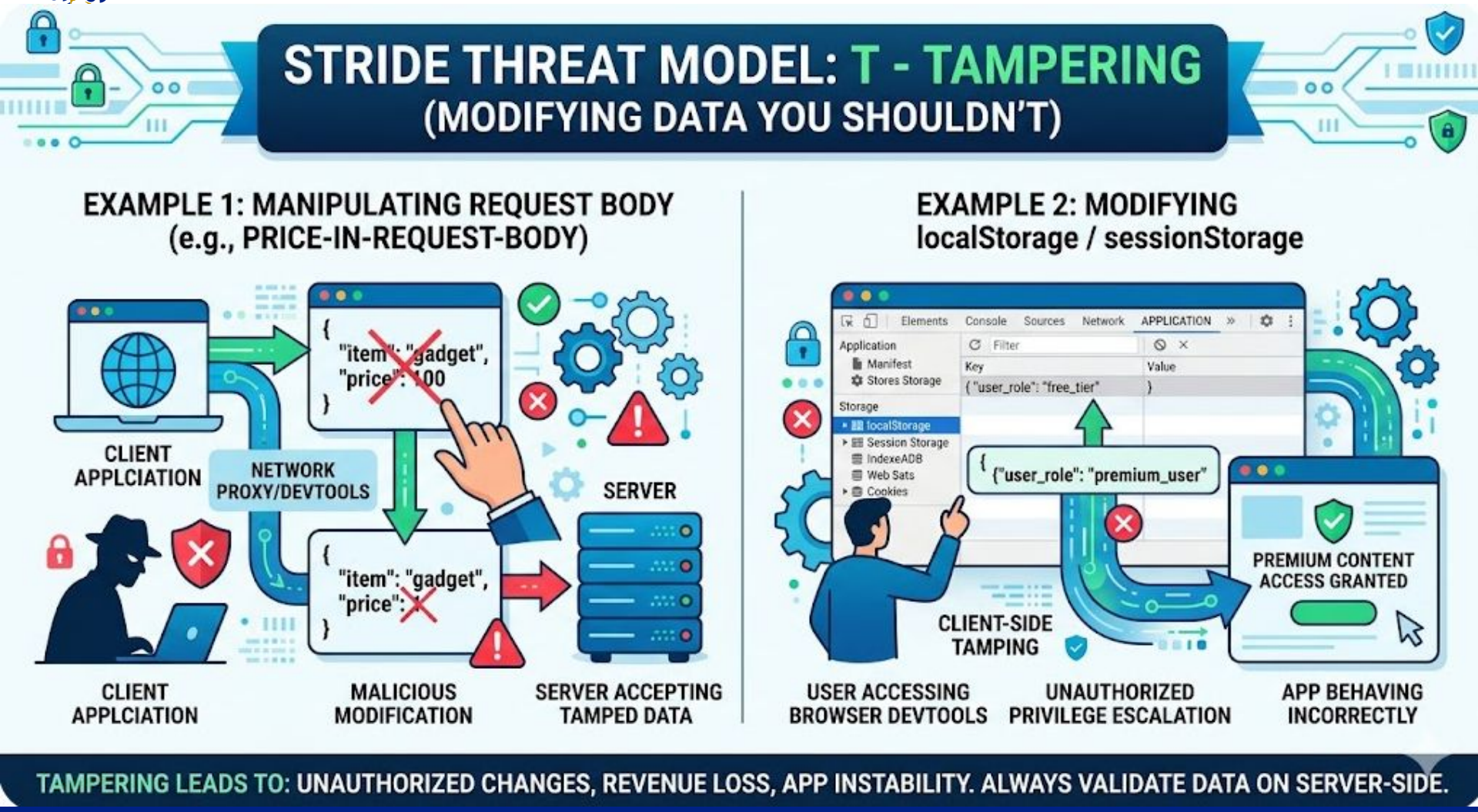
- Structured methodology
  - Identifying security risks
- Goal: not to make “unbreakable” system, but to consciously map and manage possible attack surfaces

- STRIDE
  - Spoofing
  - Tampering
  - Repuditation
  - Information disclosure
  - Denial of Service (DoS)
  - Elevation of Privilege

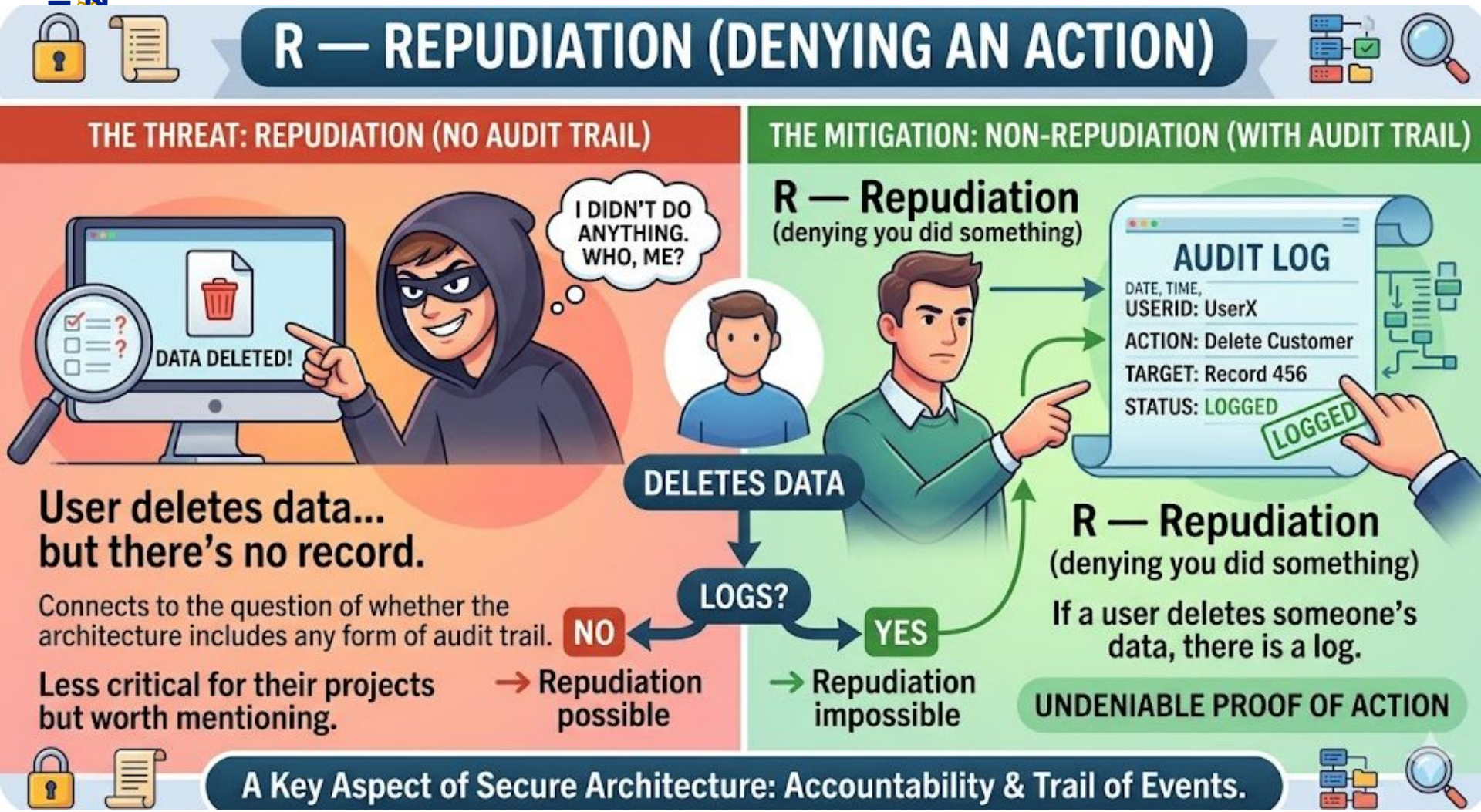
S - Spoofing


STRIDE: S — SPOOFING
(Pretending to be someone else)
PUT /users/42/profile
(Sends with other user's ID)
1. SENDS
FORGED REQUEST
MALICIOUS USER
Authenticated as
User 99
1. Authenticate Request
(User 99 valid)
2. Authorize Request
(NO CHECK:
Is User 99 == User 42?)
WEBSERVER / API ENDPOINT
SPOOFING
THREAT!
PROFILE
MODIFIED
Attacker
successfully
modifies data!
VICTIM'S PROFILE
(User 42)
PREVENTION: SECURE AUTHENTICATION IMPLEMENTATION
Verify Authenticated User matches the targeted Resource ID (User 42)

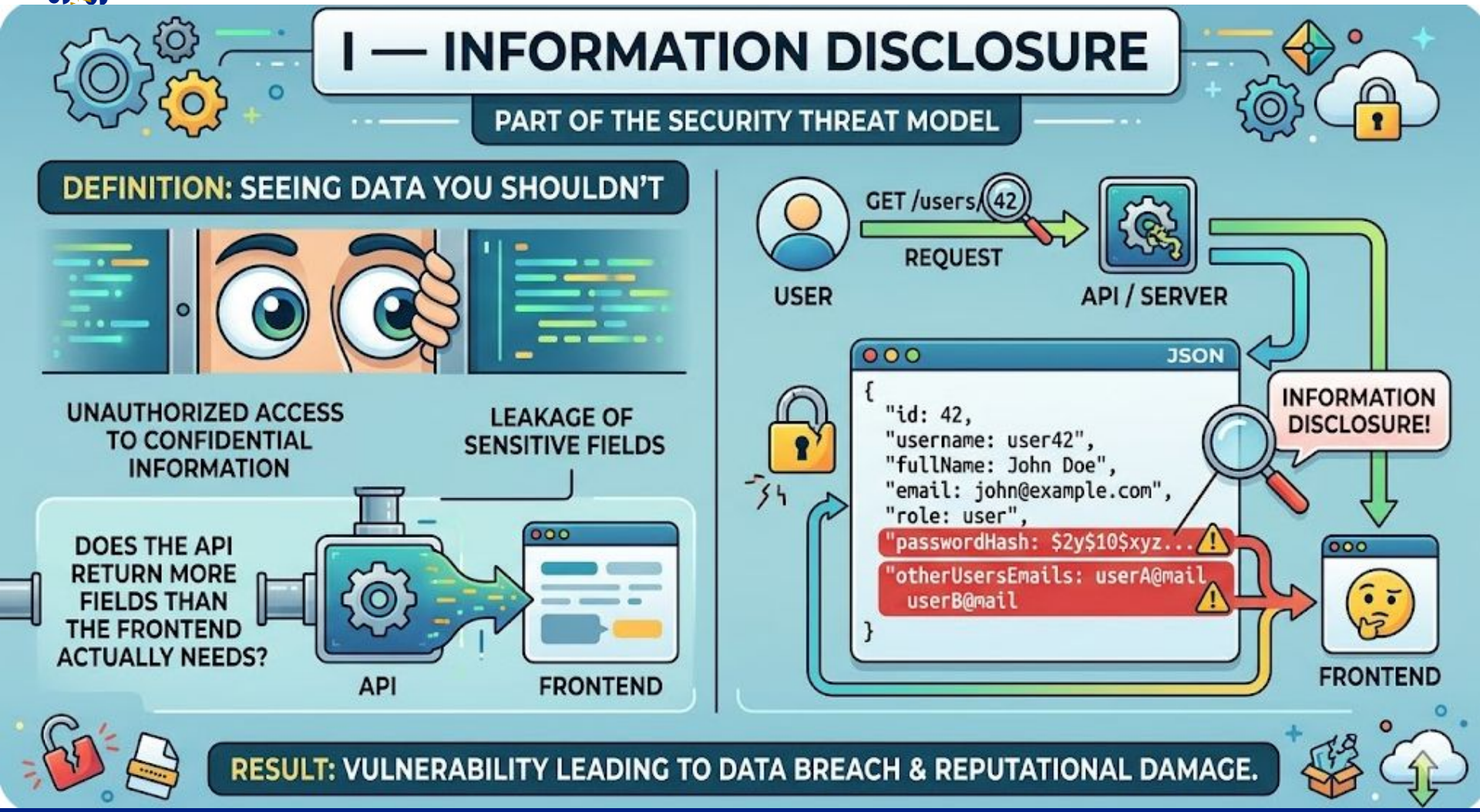
T - Tampering


STRIDE THREAT MODEL: T - TAMPERING
(MODIFYING DATA YOU SHOULDN'T)
EXAMPLE 1: MANIPULATING REQUEST BODY
(e.g., PRICE-IN-REQUEST-BODY)
{
"item": "gadget",
"price": 100
}
CLIENT APPLCIATION
NETWORK PROXY/DEVTOOLS
SERVER
{
"item": "gadget",
"price": 1
}
CLIENT APPLCIATION
MALICIOUS MODIFICATION
SERVER ACCEPTING TAMPED DATA
EXAMPLE 2: MODIFYING
localStorage / sessionStorage
Elements
Console
Sources
Network
APPLICATION
Application
Manifest
Stores Storage
Storage
localStorage
Session Storage
IndexeADB
Web Sats
Cookies
Filter
Key
Value
{ "user_role": "free_tier"
}
{ "user_role": "premium_user"
PREMIUM CONTENT ACCESS GRANTED
CLIENT-SIDE TAMPING
USER ACCESSING BROWSER DEVTOOLS
UNAUTHORIZED PRIVILEGE ESCALATION
APP BEHAVING INCORRECTLY
TAMPERING LEADS TO: UNAUTHORIZED CHANGES, REVENUE LOSS, APP INSTABILITY. ALWAYS VALIDATE DATA ON SERVER-SIDE.

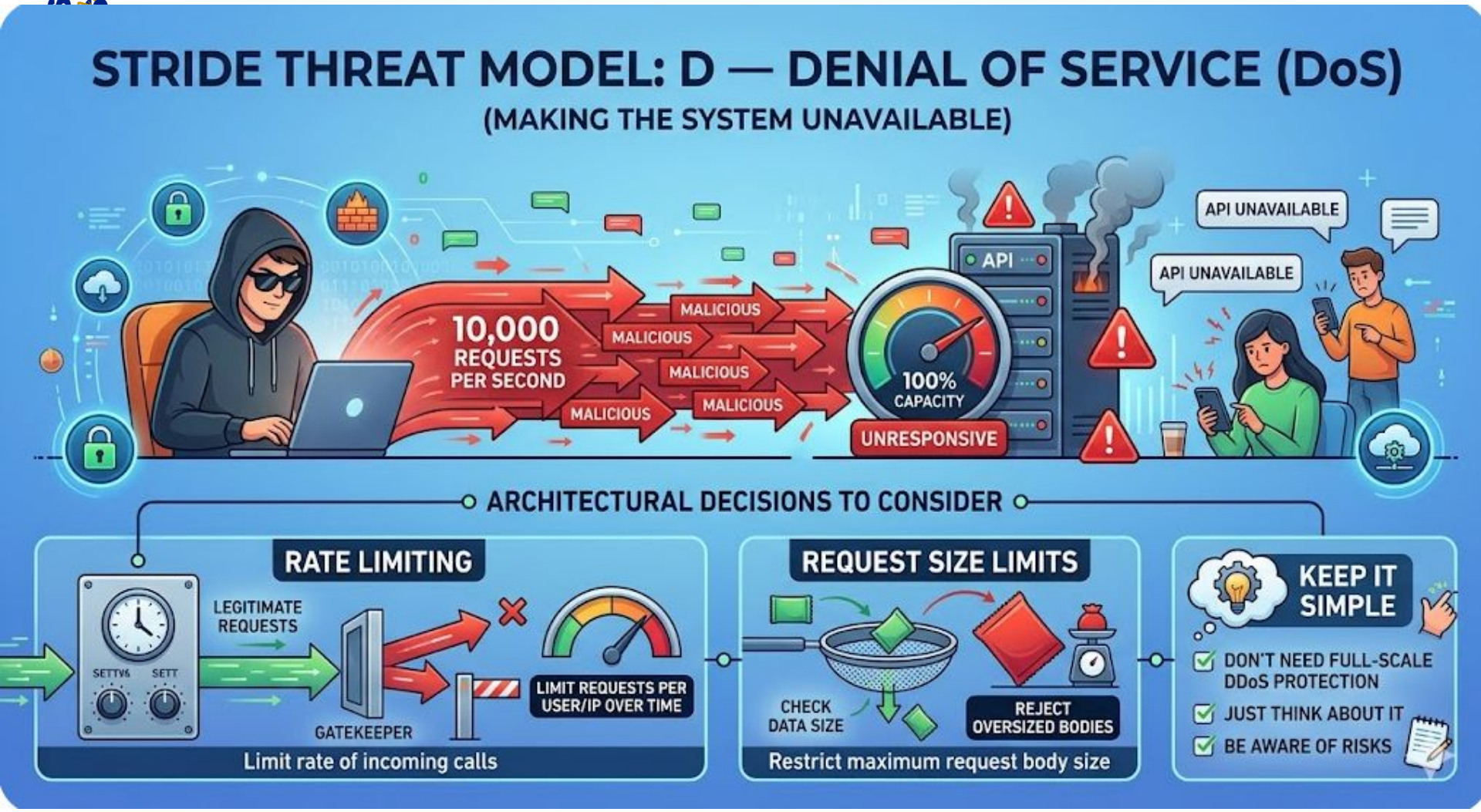
R - Repudiation



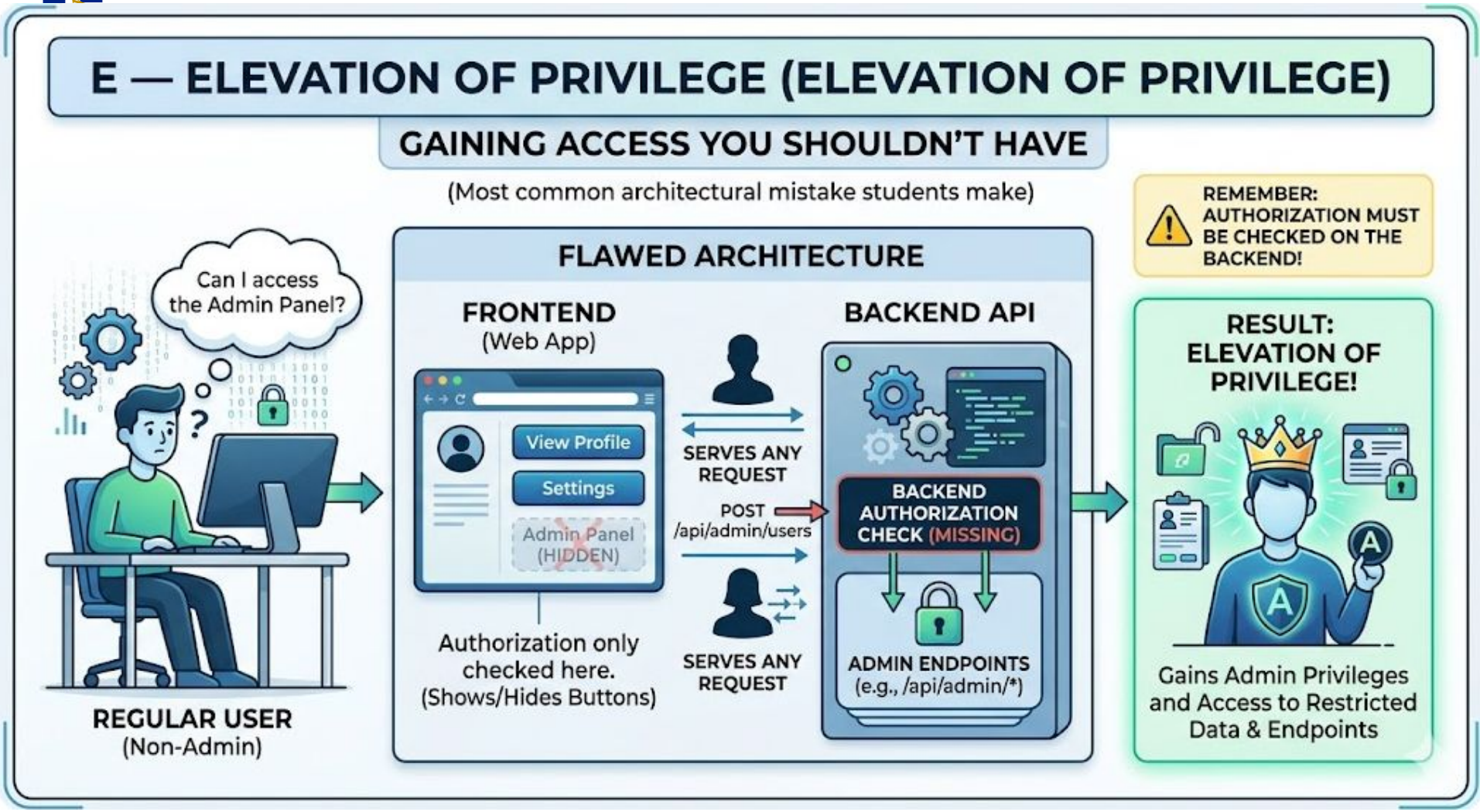
I - Information Disclosure



D - Denial of Service



E - Elevation of Privilege



From Threats to Architectural Decisions

MITIGATE

Mitigate — change the architecture or add a mechanism.

Example: add server-side validation middleware to handle tampering threats.

This is an architectural decision (where does validation live?) that deserves an ADR (Architectural Decision Record).

ACCEPT

Accept — acknowledge the risk but decide not to address it, with justification.

"We accept that a single user could make many rapid requests because our app is intended for a a small friend friend group, not the public internet."

This is also a valid architectural decision.

TRANSFER

Transfer — delegate to an external service or framework.

- **Example:** "We use bcrypt for password hashing nashing rather than implementing our own."
- **Or:** "We rely on the framework's built-in CSRF protection."

Threat Model in practice

Option A — Table format

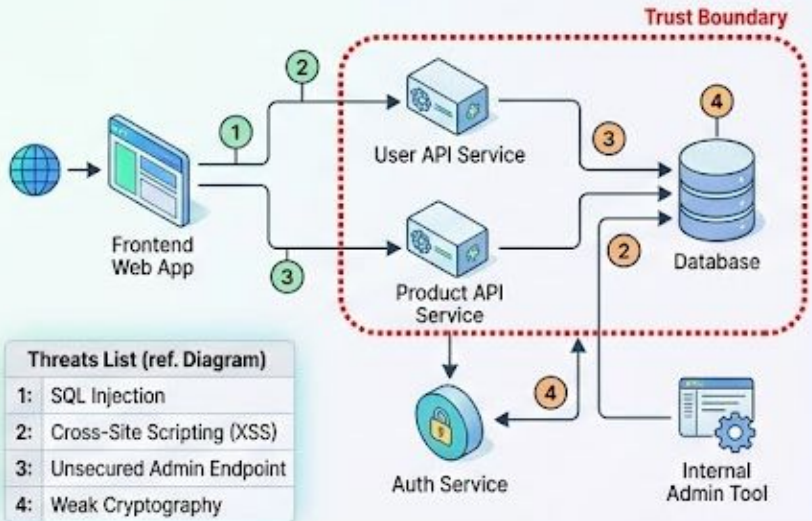
The simplest format and perfectly sufficient.

Sample Markdown table

Threat	STRIDE Category	Affected Component	Severity	Mitigation	Status
SQL Injection via login	Tampering	User API	High	Use parameterized queries	Implemented
XSS on profile page	Information Disclosure	Frontend Web App	Medium	Input sanitization	Planned
Credential stuffing	Tampering	Frontend Web App	High	Input sanitization	Accepted
Credential stuffing	Spoofing	Auth Service	High	Rate limiting / MFA	Accepted

Option B — Annotated architecture diagram

Take their existing architecture diagram and mark trust boundaries on it, with numbered threat annotations that reference a companion table.



Threats List (ref. Diagram)	
1:	SQL Injection
2:	Cross-Site Scripting (XSS)
3:	Unsecured Admin Endpoint
4:	Weak Cryptography